

Monolito mínimo con Node.js y Express

Guía de laboratorio

A continuación se presenta el “monolito mínimo” paso a paso, indicando las versiones mínimas necesarias de Node.js y npm.

1. Versiones mínimas requeridas

Para evitar errores de dependencias o funcionamiento incorrecto, se requieren las siguientes versiones:

- **Node.js** (mínimo: 16.9.0. **recomendado:** LTS, por ejemplo 18.x o 20.x.)
- **npm**: la versión que venga incluida con la instalación de Node LTS elegida (para Node 18/20 normalmente es 8.x–10.x).

Para comprobar las versiones en la terminal:

```
node -v    # deberia ser >= 16.9.0 (ideal: 18.x o 20.x)
npm -v
```

2. Crear el proyecto

1. Crear la carpeta del proyecto:

```
mkdir tienda-monolito
cd tienda-monolito
```

2. Inicializar el proyecto Node:

```
npm init -y
```

3. Instalar Express:

```
npm install express
```

Esto instalará una versión reciente de Express compatible con una versión de Node.

3. Crear index.js con el monolito mínimo

En la carpeta `tienda-monolito`, crear el archivo `index.js` con el siguiente contenido completo:

```

1      // index.js
2      // Monolito minimo con Express
3
4      const express = require('express');
5
6      const app = express();
7      const PORT = 3000;
8
9      // Middleware para poder leer JSON en el body de las peticiones
10     app.use(express.json());
11
12     // ===== Datos en memoria =====
13
14     // Catalogo de productos (normalmente esto estaria en una BD)
15     const productos = [
16       { id: 1, nombre: 'Laptop basica', precio: 12000 },
17       { id: 2, nombre: 'Mouse inalambrico', precio: 350 },
18       { id: 3, nombre: 'Teclado mecanico', precio: 1500 }
19     ];
20
21     // Pedidos registrados durante la ejecucion
22     const pedidos = [];
23
24     // ===== Rutas =====
25
26     // GET /productos -> devuelve el listado de productos
27     app.get('/productos', (req, res) => {
28       res.json(productos);
29     });
30
31     // POST /pedidos -> registra un nuevo pedido en memoria
32     app.post('/pedidos', (req, res) => {
33       const nuevoPedido = req.body;
34
35       // Validacion muy basica
36       if (
37         !nuevoPedido ||
38         !nuevoPedido.productos ||
39         !Array.isArray(nuevoPedido.productos)
40       ) {
41         return res.status(400).json({
42           mensaje: 'El pedido debe incluir un arreglo de
43             productos.'
44         });
45       }
46
47       // Asignar un id incremental
48       const id = pedidos.length + 1;
49       const pedidoConId = { id, ...nuevoPedido };
50
51       // Guardar en memoria
52       pedidos.push(pedidoConId);

```

```

53         // Responder con el pedido creado
54         res.status(201).json(pedidoConId);
55     });
56
57     // (Opcional) GET /pedidos -> ver todos los pedidos registrados
58     app.get('/pedidos', (req, res) => {
59         res.json(pedidos);
60     });
61
62     // ===== Arrancar el servidor =====
63
64     app.listen(PORT, () => {
65         console.log('Servidor escuchando en http://localhost:${PORT}');
66     });

```

Este archivo único contiene datos en memoria, rutas HTTP y el arranque del servidor, por lo que constituye un “monolito mínimo” funcional.

4. Ejecutar el servidor

Desde la carpeta del proyecto, ejecutar:

```
node index.js
```

Si todo está bien, aparecerá en la terminal:

```
Servidor escuchando en http://localhost:3000
```

El proceso se mantendrá en ejecución, indicando que el servidor está levantado.

5. Probar las rutas

5.1. GET /productos

Probar en el navegador o Postman:

```
http://localhost:3000/productos
```

Se debería mostrar el arreglo de productos en formato JSON.

5.2. POST /pedidos

Ejemplo usando curl (también puede hacerse con Postman):

Listing 1: Usando linux, Mac

```

curl -X POST http://localhost:3000/pedidos \
-H "Content-Type: application/json" \
-d "{
    \"cliente\": \"Ana Perez\",
    \"productos\": [1, 3],
    \"total\": 13500
}"

```

Listing 2: Usando cmd de Windows

```
curl -X POST "http://localhost:3000/pedidos" ^  
-H "Content-Type: application/json" ^  
-d "{  
    \"cliente\": \"Ana Perez\",  
    \"productos\": [1, 3],  
    \"total\": 13500  
}"
```

Listing 3: Usando PowerShell de Windows

```
curl -X POST "http://localhost:3000/pedidos" ^  
-H "Content-Type: application/json" ^  
-d '{  
    "cliente": "Ana Perez",  
    "productos": [1, 3],  
    "total": 13500  
'
```

La respuesta esperada es un JSON similar a:

```
{  
    "id": 1,  
    "cliente": "Ana Perez",  
    "productos": [1, 3],  
    "total": 13500  
}
```

5.3. GET /pedidos (opcional)

Para ver todos los pedidos registrados durante la ejecución:

```
http://localhost:3000/pedidos
```

Esto mostrará el arreglo `pedidos` con todos los pedidos creados en memoria.